Comparative Analysis of Multilayer Perceptron (MLP) and Linear Support Vector Machine (LinearSVC) Architectures for Multiclass Image Classification on CIFAR-10

Furkan Berkay KILIC
Department of Engineering Science
University West (Högskolan Väst)
Trollhättan, Sweden
fuki0001@student.hv.se

Mert SAHINER
Department of Engineering Science
University West (Högskolan Väst)
Trollhättan, Sweden
mesa0014@student.hv.se

Ilker GUNGOR
Department of Engineering Science
University West (Högskolan Väst)
Trollhättan, Sweden
ilgu0002@student.hv.se

David SALMO
Department of Engineering Science
University West (Högskolan Väst)
Trollhättan, Sweden
dasa0023@student.hv.se

***Abstract*— This study presents a comparative evaluation of two supervised machine learning paradigms—a feedforward Multilayer Perceptron (MLP) neural network and a classical Linear Support Vector Machine (LinearSVC)—applied to the 10-class CIFAR-10 image classification benchmark. To ensure strict experimental reproducibility and prevent data leakage, the original 50,000 training images were partitioned via stratified sampling (random_state=42) into 45,000 training and 5,000 validation samples, while reserving the standard 10,000-image test set for final evaluation. Channel-wise RGB normalization statistics were computed exclusively from the training partition and shared across both pipelines via a modular data loader. Both models process flattened 3,072-dimensional pixel feature vectors derived from the $32 \times 32 \times 3$ input images. For the LinearSVC model, a multi-stage hyperparameter search across 2,000-sample and 10,000-sample stratified subsets identified an optimal regularization parameter of $C = 0.0001$, achieving a final test accuracy of 40.70% and a macro-averaged F1-score of 0.39 when trained on all 45,000 samples. In contrast, the single-hidden-layer MLP architecture $(3072 \rightarrow 128 \rightarrow 10)$ utilizing a non-linear ReLU activation function achieved a test accuracy of 50.21% and a macro-averaged F1-score of 0.49, outperforming the Linear SVM by 9.51 percentage points across all ten object categories.**

I. Introduction

Multiclass image classification is a foundational problem in applied artificial intelligence that requires learning discriminative representations from high-dimensional raw pixel inputs. The CIFAR-10 dataset provides a standardized benchmark comprising 60,000 color images of size $32 \times 32 \times 3$ distributed evenly across ten mutually exclusive categories: *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship*, and *truck*.

Within the scope of classical machine learning and introductory neural networks, two distinct approaches are commonly employed on flattened image vectors: margin-based linear classifiers and multilayer neural networks. Support Vector Machines (SVMs) seek an optimal decision hyperplane that maximizes the margin between classes in the input feature space, governed by the regularization hyperparameter C. Conversely, Multilayer Perceptrons (MLPs) project the input vector through intermediate hidden layers equipped with non-linear activation functions such as the Rectified Linear Unit (ReLU), enabling the network to model non-linear interactions among pixel intensities. This report systematically compares an MLP classifier against a LinearSVC model under identical data preprocessing conditions, evaluating both overall test metrics and per-class error patterns.

A. Problem Statement

Image classification is a fundamental task in machine learning that aims to assign input images to predefined categories. This project examines the classification of small Red-Green-Blue (RGB) color images from the CIFAR-10 dataset into ten distinct classes. Several categories exhibit high visual similarity, and the low spatial resolution of 32 by 32 pixels blurs fine details, making accurate recognition challenging for classification models operating on flattened pixel vectors.

B. Motivation

The motivation for this study is to examine how different machine learning paradigms perform on the exact same visual classification task when all data preparation steps are held constant. Specifically, comparing a Multilayer Perceptron (MLP) and a Linear Support Vector Machine (LinearSVC) demonstrates the practical difference between learning non-linear intermediate feature representations via hidden layers and constructing direct linear maximum-margin decision boundaries in high-dimensional pixel space.

C. Background and Context

Both classical margin classifiers and artificial neural networks are widely used for pattern recognition tasks. When processing flattened image vectors of 3,072 pixel values, LinearSVC attempts to separate each class from all others using linear hyperplanes, where the regularization parameter C balances margin width against training errors. In contrast, a standard MLP processes flattened image pixels through fully connected hidden layers followed by non-

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

linear ReLU activations, allowing the model to capture non-linear combinations of pixel intensities. Comparing these two architectures provides clear insight into the advantages of non-linear hidden representations over purely linear classifiers for computer vision applications.

II. Methodology

A. Dataset and Data Source

The CIFAR-10 data were obtained programmatically through the torchvision.datasets library. The dataset consists of 60,000 color images of size 32×32 pixels across three RGB channels (torch.Size([3, 32, 32])), evenly divided into ten classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The official split provides 50,000 training images and 10,000 test images. Representative examples from each category are shown in Fig. 1.

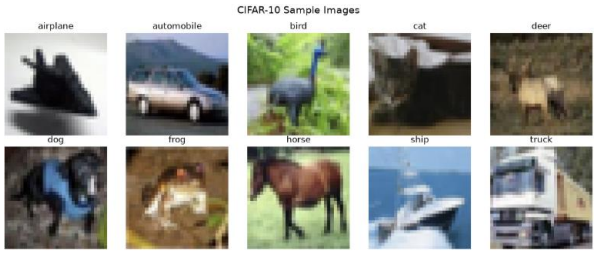


Fig. 1. CIFAR-10 Sample Images

B. Data Preprocessing

- To ensure reproducibility across group experiments, a shared data preparation module (data_preparation.py) was created. First, the original 50,000 training data were partitioned into 45,000 training images (90%) and 5,000 validation images (10%) using stratified sampling with a fixed random seed of 42. Stratification guaranteed that each class retained exactly 4,500 training and 500 validation samples. The split indices were saved in config/split_indices.npz.

- Second, raw pixel intensities were converted to 32-bit floating-point PyTorch tensors scaled to $[0,1]$. To prevent data leakage from the validation or test sets, channel-wise normalization statistics were calculated exclusively from the 45,000 training images, yielding a mean of $[0.4911,0.4821,0.4464]$ and a standard deviation of $[0.2469,0.2434,0.2616]$. These parameters were stored in config/preprocessing_config.json and applied identically to standardize the training, validation, and test partitions to zero mean and unit variance.

C. Exploratory Data Analysis

Exploratory Data Analysis (EDA) confirmed that CIFAR-10 maintains a completely balanced distribution with 5,000 training images and 1,000 test images per category, as illustrated in Fig. 2. Visual inspection of the dataset revealed that objects appear at varying scales and viewpoints, and several categories share background colors (for example, blue sky or water in airplane, bird, and ship images, and green foliage in deer, horse, and frog images), which complicates classification when spatial shapes are not captured.

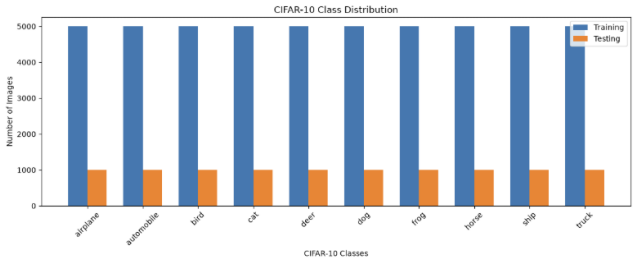


Fig. 2. CIFAR-10 Class Distribution

D. Artificial Intelligence and Machine Learning Models

- Two supervised classification models were implemented within the course scope, as summarized in Table I. The baseline MLP flattens each 3 times 32 times 32 input image into a one-dimensional (1D) vector of 3,072 features, processes it through a single fully connected hidden layer with 128 neurons and a Rectified Linear Unit (ReLU) activation function, and maps the result to a 10-neuron linear output layer (394,634 total trainable parameters).The LinearSVC model also receives the flattened 1D vector of 3,072 normalized pixel features (converted from the PyTorch DataLoaders into 2D NumPy arrays) and learns ten one-vs-rest linear decision hyperplanes. Regularization is controlled by the hyperparameter $C = 0.0001$ (max_iter=3000, dual="auto"), which penalizes complex weights and widens the classification margin to prevent overfitting in the 3,072-dimensional feature space (3,072 times 10 + 10 = 30,730 total learned weights and intercepts).

- TABLE I. **Architectural Summary of Implemented Models**

Component	Baseline MLP	Linear SVM (LinearSVC)
Input Format	Flattened 1D vector (3,072)	Flattened 1D vector (3,072)
Feature Transformation	1 Linear hidden layer (128, ReLU)	Direct input space (Linear kernel)
Classifier Head	Linear ($128 \rightarrow 10$ logits)	One-vs-Rest Linear Hyperplanes (10 classes)
Regularization / Control	Best validation checkpoint (`Epoch 7`)	L_2 margin regularization ($C = 0.0001$)
Spatial Structure	Lost during flattening	Lost during flattening
Parameters	394,634	30,730

E. Method Selection Rationale

The MLP was selected as a neural network baseline due to its architectural simplicity, representing how a dense feedforward network with non-linear ReLU activations learns intermediate representations from raw pixel vectors. The LinearSVC model was chosen from the classical machine learning curriculum to evaluate how a maximum-margin linear classifier performs on the exact same high-dimensional feature space while remaining computationally tractable on 45,000 training images.

III. Experimental Setup

A. Tools, Languages, and Libraries

The project was implemented in Python (v3.14) using PyTorch (v2.15.0) and Torchvision (v0.29.2) for data loading and neural network development, NumPy for array manipulation, Scikit-learn (LinearSVC, train_test_split, and evaluation metrics) for classical machine learning, and Matplotlib for visualization within Jupyter Notebook..

B. Development Environment

Experiments were conducted locally within a dedicated Python/Anaconda environment on local macOS and Windows workstations. All data extraction, progressive SVM hyperparameter searches, model training, and final test evaluations were executed on the Central Processing Unit (CPU).

C. Data Splitting and Validation Strategy

Both models used the exact same data splits (45,000 training, 5,000 validation, and 10,000 testing images) and identical channel-wise normalization loaded via data_preparation.py. For the MLP, training was conducted for 10 epochs (batch_size=64) using Cross-Entropy loss and the Adam optimizer with a learning rate of 0.001. Validation accuracy was monitored after each epoch, peaking at **51.20% in Epoch 7** (while training accuracy reached 61.43% by Epoch 10), and the weights achieving the highest validation accuracy were restored and saved to mlp_weights.pth.

For the LinearSVC model, DataLoader batches were reshaped into 2D NumPy matrices of shape (45000, 3072) for training, (5000, 3072) for validation, and (10000, 3072) for testing. To tune the regularization parameter C efficiently, a progressive validation strategy was executed:

1. **2,000-Sample Subset:** Testing $C = 1.0$ (22.96% val accuracy, 54.01 s), $C = 0.01$ (26.12% val accuracy, 74.75 s), and $C = 0.001$ (31.04% val accuracy, 16.86 s, 168 iterations) showed that stronger regularization significantly improved both convergence speed and generalization.
2. **10,000-Sample Subset:** Scaling to 10,000 stratified training samples and testing $C = 0.001$ (36.28% val accuracy, 135.21 s), $C = 0.0001$ (**38.88%** val accuracy, 72.37 s, 6 iterations), and $C = 0.00001$ (37.52% val accuracy, 39.23 s) identified $C = 0.0001$ as the optimal parameter.
3. **Full 45,000-Sample Training:** Training the final LinearSVC model on all 45,000 samples with $C = 0.0001$ (max_iter=3000, dual="auto") converged in 616.90 seconds (6 iterations per class) and achieved a validation accuracy of **41.10%**.

IV. Results and Discussion

A. Performance Evaluation Metrics

Both models were evaluated on the unseen 10,000-image CIFAR-10 test set. Because multiclass classification requires assessing both false positives and false negatives, we evaluated overall Accuracy, macro-averaged Precision (proportion of correct positive predictions), macro-averaged Recall (proportion of actual class samples correctly detected), and macro-averaged F1-score (the harmonic mean of Precision and Recall). As shown in Table II, the MLP achieved a **9.51 percentage-point improvement** in test accuracy over the LinearSVC model (50.21% vs. 40.70%).

TABLE II. **Test Set Performance Comparison on 10,000 Unseen Images**

Model	**Accuracy (%)**	**Precision (Macro)**	**Recall (Macro)**	**F1-Score (Macro)**
LinearSVC ($C = 0.0001$)	40.70	0.40	0.41	0.39
MLP (3072 → 128 → 10)	**50.21**	**0.50**	**0.50**	**0.49**
Improvement (MLP over SVM)	**+9.51**	**+0.10**	**+0.09**	**+0.10**

TABLE III. **Per-Class Test Set F1-Score, Precision, and Recall Comparison (10,000 Images)**

Class	MLP Precision	MLP Recall	MLP F1-Score	SVM Precision	SVM Recall	SVM F1-Score
airplane	0.61	0.54	0.57	0.44	0.50	0.47
automobile	0.63	0.57	0.60	0.41	0.52	0.46
bird	0.38	0.45	0.42	0.37	0.22	0.27
cat	0.35	0.19	0.25	0.30	0.18	0.22
deer	0.46	0.32	0.38	0.40	0.26	0.31
dog	0.40	0.44	0.42	0.36	0.32	0.34
frog	0.50	0.61	0.55	0.39	0.53	0.45
horse	0.52	0.63	0.57	0.44	0.47	0.46
ship	0.58	0.69	0.63	0.46	0.57	0.51
truck	0.53	0.57	0.55	0.40	0.49	0.44

B. Key Findings

Inspection of the per-class metrics in Table III and the confusion matrices for the MLP and LinearSVC models indicates that both architectures experienced the greatest difficulty with visually similar animal categories, particularly *cat*, *bird*, *deer*, and *dog*. In the LinearSVC model, *cat* achieved the lowest F1-score (0.22, with only 18.00% recall and 30.00% precision), followed by *bird* (0.27 F1-score, 22.00% recall), *deer* (0.31 F1-score, 26.00% recall), and *dog* (0.34 F1-score, 32.00% recall). The LinearSVC model achieved its highest F1-scores on *ship* (0.51 F1-score, 57.00% recall), *airplane* (0.47 F1-score, 50.00% recall), *automobile* (0.46 F1-score, 52.00% recall), and *horse* (0.46 F1-score, 47.00% recall).

In contrast, the MLP improved recognition across all ten categories. Specifically, the MLP boosted *bird* recognition from 0.27 to 0.42 F1-score (45.00% recall), *automobile* from 0.46 to 0.60 F1-score (63.00% precision, 57.00% recall), *ship* from 0.51 to 0.63 F1-score (69.00% recall), and *horse* from 0.46 to 0.57 F1-score (63.00% recall). Nevertheless, *cat* remained the most challenging category for the MLP as well (0.25 F1-score, 19.00% recall), frequently being misclassified as *dog*, *frog*, or *bird*.

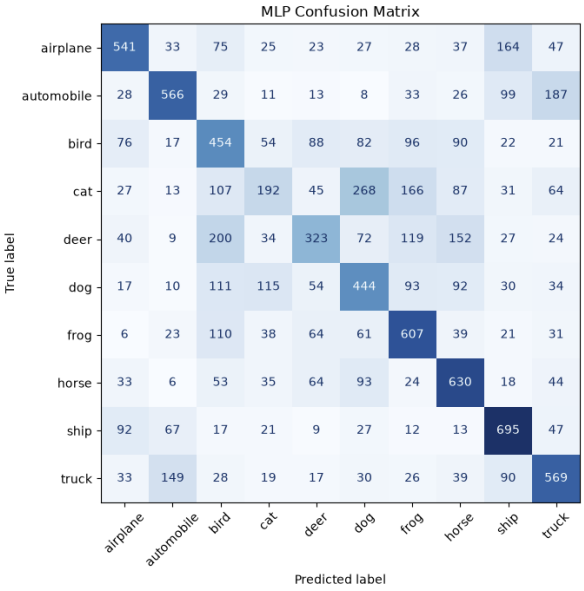


Fig. 4. MLP – CIFAR-10 Test Confusion Matrix

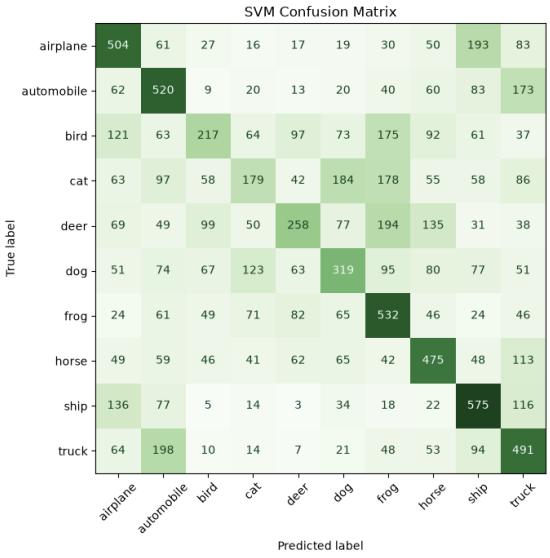


Fig. 5. LinearSVC – CIFAR-10 Test Confusion Matrix

C. Result Interpretation

The 9.51 percentage-point performance advantage of the MLP over LinearSVC (50.21% vs. 40.70%) is directly linked to how each architecture models decision boundaries in high-dimensional pixel space. A LinearSVC classifier is restricted to finding a single linear hyperplane per class based on a weighted sum of the 3,072 pixel intensities. Because objects in CIFAR-10 vary in position, lighting, and background, pixel intensities are not linearly separable. By passing the 3,072 inputs through a 128-neuron hidden layer with non-linear ReLU activations, the MLP learns non-linear combinations of pixel features, allowing it to separate overlapping classes much more effectively.

At the same time, both models exhibit an overall accuracy ceiling around 40–50% because flattening the $32 \times 32 \times 3$ image into a 1D vector destroys 2D spatial neighborhoods, causing both classifiers to rely heavily on global background colors (such as blue water/sky for *ship* and *airplane*) rather than local object shapes.

V. Conclusion

A. Summary of Findings

This study compared a Multilayer Perceptron (MLP) and a Linear Support Vector Machine (LinearSVC) on the CIFAR-10 dataset under identical preprocessing and data partitioning conditions. Through progressive validation tuning, strong regularization ($C = 0.0001$) enabled the LinearSVC model to achieve 40.70% test accuracy and a 0.39 macro F1-score. The MLP achieved a test accuracy of 50.21% and a macro F1-score of 0.49, outperforming the LinearSVC baseline by 9.51 percentage points.

B. Lessons Learned

We learned that introducing non-linear hidden layers (MLP with ReLU) is substantially more effective for multi-class image recognition on flattened pixels than relying on purely linear decision hyperplanes (LinearSVC). Furthermore, conducting progressive hyperparameter tuning on smaller stratified subsets (2,000 and 10,000 samples) proved essential for identifying the optimal SVM regularization parameter (C=0.0001) before scaling to 45,000 training images, while sharing fixed split indices (split_indices.npz) and training-only normalization parameters (preprocessing_config.json) guaranteed a leak-free workflow.

C. Limitations

This study was limited to foundational architectures operating on flattened 1D pixel vectors without data augmentation to remain within the course scope. Flattening discards 2D spatial structure, which limits accuracy on deformable classes like *cat* and *dog*. Additionally, while non-linear kernel SVMs (such as Radial Basis Function kernels) could theoretically capture non-linear boundaries, their quadratic-to-cubic training complexity makes them computationally prohibitive on 45,000 high-dimensional (3,072-feature) images on standard CPU hardware.

D. Future Work

Future improvements include applying dimensionality reduction techniques such as Principal Component Analysis (PCA) so that non-linear RBF-kernel SVMs can be trained efficiently, as well as exploring deeper multilayer neural networks with dropout regularization, batch normalization, and data augmentation (random horizontal flips and crops) to further improve separation between visually similar classes.

Acknowledgment

During this project, the authors utilized generative artificial intelligence tools, including ChatGPT and Google Gemini, to assist with code debugging, pipeline organization, and language editing. All experimental runs, model training, evaluation metrics, and figures were executed and verified directly by the group members.

References

[1] A. Krizhevsky, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[2] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035. [Online]. Available: https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011

[4] F. B. Kilic, M. Sahiner, I. Gungor, and D. Salmo, "CIFAR10-Image-Classification," GitHub repository, 2026. [Online]. Available: https://github.com/Davidsalmo792/CIFAR10-Image-Classification